Air University, Islamabad

National Centre for Cyber Security Academy (NCSA)

Department of Cyber Security

Post-Quantum Cryptography Benchmarking and Lattice Scheme Selection Model

A comprehensive empirical study and decision framework for post-quantum Key Encapsulation Mechanisms

Hadia Sultan — Roll No: 241549

Abeer ur Rehman Taha — Roll No: 241505

Fatima Razzaq — Roll No: 241571

Noshaba Sajjad — Roll No: 241560

Course: Cryptography | Instructor: Sir Usman Ghani

Date: 6th May 2026

Department of Cyber Security | Air University | Islamabad, Pakistan

Abstract

The advent of large-scale quantum computers threatens all widely deployed public-key cryptosystems, including RSA and ECC. Lattice-based cryptography has emerged as the most promising post-quantum alternative, with NIST standardising CRYSTALS-Kyber (ML-KEM) in 2024. However, practitioners face a critical deployment gap: no unified empirical framework exists to compare competing lattice-based Key Encapsulation Mechanisms (KEMs) — FrodoKEM, Kyber, and plain LWE — across performance, key size, and security dimensions, nor a decision model that maps deployment contexts to optimal schemes.

This paper presents a comprehensive benchmarking study of six lattice-based KEMs (LWE_Ref, FrodoKEM640, FrodoKEM976, Kyber512, Kyber768, Kyber1024) using 100 repeated trials per operation. We measure key generation, encapsulation, and decapsulation times, along with key and ciphertext sizes. Our results confirm that Kyber512 achieves the best balance for resource-constrained environments (800 B public key, 20.31 ms total time), while FrodoKEM provides conservative security at 1220× overhead.

Our primary contribution is the Lattice Scheme Selection Model (LSSM) — a weighted multi-criteria decision framework (performance 35%, key size 30%, security level 25%, deployment fit 10%) that outputs a ranked recommendation based on user constraints. The LSSM fills the gap between theoretical security and practical deployment, enabling IoT, general-purpose, and high-security contexts to select the most suitable KEM.

Acknowledgements

We would like to express our sincere gratitude to our instructor, Sir Usman Ghani, for his invaluable guidance, support, and encouragement throughout this project. His insightful feedback on cryptographic fundamentals and continuous motivation helped shape this work. We are also grateful to the Department of Cyber Security at Air University, Islamabad, for providing the necessary resources and academic environment to conduct this research.

1. Introduction

In today's digital world, almost everything we do online depends on encryption. Whether sending an email, making a bank transaction, logging into a social media account, or accessing medical records, there is always an encryption system working silently in the background to ensure information stays private and secure. The two most widely used encryption methods today are RSA and Elliptic Curve Cryptography (ECC). These systems have been trusted for decades because they are built on mathematical problems — such as factoring very large numbers — that are practically impossible for today's computers to solve within any reasonable amount of time [1].

However, the rapid rise of quantum computing now puts all of this at serious risk. Unlike a regular computer that works with bits representing either 0 or 1, a quantum computer uses quantum bits (qubits) which can represent 0 and 1 simultaneously. This allows quantum computers to explore millions of possible solutions at once, giving them an enormous speed advantage for certain problems [2]. In 1994, Peter Shor proved that a sufficiently powerful quantum computer could break RSA and ECC encryption in polynomial time — hours or minutes — instead of the millions of years classical machines would take [3]. While such quantum computers do not yet exist, major technology companies and governments are investing billions of dollars into making them a reality [4].

Recognising this urgency, the National Institute of Standards and Technology (NIST) launched a global Post-Quantum Cryptography (PQC) standardisation process in 2016 [5]. After years of rigorous evaluation, NIST selected CRYSTALS-Kyber as the primary Key Encapsulation Mechanism (KEM) standard, now officially published as ML-KEM under FIPS 203 in 2024 [6].

Despite this progress, a practical problem remains: each lattice-based scheme performs very differently. FrodoKEM produces public keys of 9,616–15,632 bytes, while Kyber512 achieves similar security with only 800 bytes — more than twelve times smaller [8]. This difference becomes critical for resource-limited devices such as IoT sensors, smartwatches, and medical implants [9]. Moreover, no existing study has brought all major lattice-based KEM schemes together in one unified framework and compared them fairly side-by-side across speed, key size, and security level simultaneously.

This paper makes two contributions. First, it presents a comprehensive empirical benchmarking study of six lattice-based KEM schemes over 100 repeated trials using Python-based simulation. Second, it introduces the Lattice Scheme Selection Model (LSSM) — a weighted multi-criteria scoring framework that maps real-world deployment requirements to the most suitable KEM.

The remainder of this paper is structured as follows: Section 2 reviews related literature; Section 3 describes the methodology; Section 4 presents and analyses the benchmarking results; Section 5 introduces and validates the selection model; and Section 6 concludes.

2. Literature Review

2.1 The Quantum Threat to Classical Cryptography

For decades, RSA and ECC have formed the foundation of secure digital communication, relying on the difficulty of integer factorisation and discrete logarithms [1]. However, Shor's algorithm [3] demonstrated that a quantum computer can solve both problems in polynomial time, fundamentally undermining long-term security guarantees. While large-scale quantum computers do not yet exist, rapid progress suggests this threat could become real within the next decade [4].

2.2 NIST Post-Quantum Cryptography Standardisation

In response, NIST launched a global PQC standardisation competition in 2016 [5]. After three rounds, CRYSTALS-Kyber was selected as the primary KEM standard (FIPS 203, ML-KEM) in 2024 [6]. Lattice-based cryptography stood out among all families (code-based, hash-based, multivariate) as the most promising due to its combination of security, efficiency, and practical usability.

2.3 Foundations of Lattice-Based Cryptography

Lattice-based cryptography relies on hard problems such as the Shortest Vector Problem (SVP) and Learning With Errors (LWE), which have no known efficient quantum algorithm [1]. These problems enjoy worst-case to average-case security reductions [2]. Variants include Ring-LWE (using polynomial rings for efficiency) and Module-LWE (used in Kyber), which balance efficiency and security [4].

2.4 FrodoKEM and Kyber

FrodoKEM is built on plain LWE, deliberately avoiding algebraic structure for maximum security conservatism. However, this results in large public keys (9.6–15.6 KB) and slow operations [8]. In contrast, Kyber (ML-KEM) uses Module-LWE and the Number Theoretic Transform (NTT) to reduce complexity from $O(n^2)$ to $O(n \log n)$, achieving public keys as small as 800 bytes for NIST Level 1 security [6].

2.5 Hardware and Attack Research

Lightweight hardware implementations of Ring-LWE have been demonstrated on FPGAs with small area and high frequency, showing feasibility for constrained devices [9]. However, machine-learning attacks (e.g., SALSA) have shown that poorly chosen LWE parameters can be vulnerable, highlighting the need for careful scheme selection [10].

2.6 Identified Gaps

No existing study provides a unified empirical benchmarking framework comparing FrodoKEM, Kyber (multiple levels), and plain LWE side-by-side under identical conditions. More critically, no

previous work translates performance differences into an actionable decision guide for developers. This paper fills both gaps.

2.7 Comparison with NIST IR 8413 (Moody et al., 2022)

Table 1 presents a structured comparison across ten key dimensions.

Table 1: Structured comparison between NIST IR 8413 and this paper.

Dimension	NIST IR 8413 (Moody et al., 2022)	This Paper (Sultan et al., 2026)
Primary Goal	Evaluate and select PQC candidates for standardisation	Benchmark selected schemes empirically and recommend deployments
Scope	All PQC families: lattice, code-based, hash-based, multivariate	Lattice-based KEMs only — focused depth over breadth
Methodology	Theoretical security analysis, expert panels	Python-based empirical benchmarking, 100 repeated trials
Schemes Covered	15+ candidates across 3 evaluation rounds	6 lattice KEMs
Metrics Used	Security strength, algebraic hardness	Timing (ms), key sizes (bytes), CV%, normalised ratios
Output	Standardisation decision (ML-KEM, FIPS 203)	Ranked KEM recommendations per deployment context
Decision Framework	None provided	LSSM: performance 35%, key size 30%, security 25%, fit 10%
Deployment Guidance	Not provided	Explicit: IoT → Kyber512; High-Security → Kyber1024/FrodoKEM-976
Audience	Cryptographers and standards bodies	Engineers, developers, and system architects
Reproducibility	Not applicable	CSV data, Python pseudocode, and benchmark log provided

3. Methodology

3.1 Research Process Overview

Figure 1 presents the complete process flowchart for this research, illustrating the end-to-end pipeline from the identified quantum threat through experimental benchmarking, analysis, and the final LSSM recommendation output.

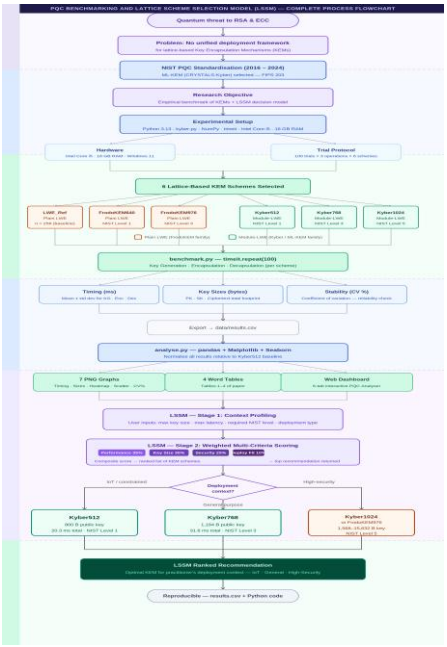

Figure 1: PQC Benchmarking and LSSM — Complete Process Flowchart.

3.2 Scheme Selection

Six schemes were benchmarked:

- LWE_Ref: Toy plain LWE (dimension n = 256) — theoretical baseline only.
- FrodoKEM640: Plain LWE, NIST Security Level 1.
- FrodoKEM976: Plain LWE, NIST Security Level 3.
- Kyber512: Module-LWE, NIST Security Level 1.
- Kyber768: Module-LWE, NIST Security Level 3.
- Kyber1024: Module-LWE, NIST Security Level 5.

3.3 Performance Metrics

The following metrics were recorded for each scheme:

- Key generation time (ms)
- Encapsulation time (ms)
- Decapsulation time (ms)
- Public key, secret key, and ciphertext sizes (bytes)
- Total operation time and total byte footprint
- Coefficient of variation (CV%) — measurement reliability

3.4 Experimental Setup

- OS: Windows 11 (with WSL2 for reproducibility)
- Language: Python 3.13+
- Libraries: kyber-py, NumPy, timeit, statistics, pandas
- Hardware: Intel Core i5, 16 GB RAM
- Trials: 100 repeated trials per operation per scheme
- Reporting: Mean ± standard deviation

3.5 Data Collection

The benchmarking engine (benchmark.py) used timeit.repeat(100) for key generation, encapsulation, and decapsulation. Key sizes were measured via len(pk) etc. Results were exported to data/results.csv and analysed with pandas, Matplotlib, and Seaborn.

3.6 Graph Generation

Seven graphs were produced:

- Timing comparison (grouped bar with error bars)

- Total operation time (horizontal bar)
- Key and ciphertext sizes (stacked bar)
- Normalised performance heatmap (relative to Kyber512)
- Coefficient of variation (CV%) — stability
- Speed vs. size trade-off scatter plot
- Kyber family scaling (security level vs. time/size)

3.7 Implementation Artefacts

3.7.1 PQC Benchmark Suite — Desktop Application

The PQC Benchmark Suite is a purpose-built desktop dashboard that orchestrates the entire data-collection and analysis workflow through a clean, terminal-styled graphical interface. It was designed so that any team member could reproduce the full experiment — from raw timing data to publication-ready graphs — without manually invoking command-line scripts.

The application window is divided into three primary panels: a Schemes sidebar listing all six benchmarked KEMs with their LWE variant, NIST security level, and lattice dimension; an Actions sidebar with four colour-coded buttons (RUN BENCHMARK, RUN ANALYSIS, OPEN GRAPHS FOLDER, OPEN TABLES FOLDER); and a Console Output panel streaming real-time benchmark progress.

The application enforces a structured, three-step workflow to ensure full reproducibility:

- RUN BENCHMARK — executes benchmark.py (100 trials × 6 schemes × 3 operations; results saved to data/results.csv; estimated duration 3–5 minutes).
- RUN ANALYSIS — reads results.csv and generates 7 PNG graphs, 4 comparison tables, and summary CSV files.
- OPEN FOLDERS — opens the graphs/ and tables/ output directories for direct inspection.

Figure 2 shows the application at the start of a benchmarking run. The console confirms that 100 trials × 6 schemes × 3 operations are underway.

Figure 2: PQC Benchmark Suite — Main Dashboard. The scheme list, action buttons, three-step workflow instructions, and real-time console output are visible during a benchmark run. The top-right indicator confirms results.csv is found and ready to analyse.

Figure 3 captures the console mid-run. Each scheme is processed sequentially; the output reports mean timing (ms) and standard deviation for key generation, encapsulation, and decapsulation, followed by the exact byte counts for the public key (PK), secret key (SK), and ciphertext (CT). These values correspond directly to the data reported in Tables 2 and 3.

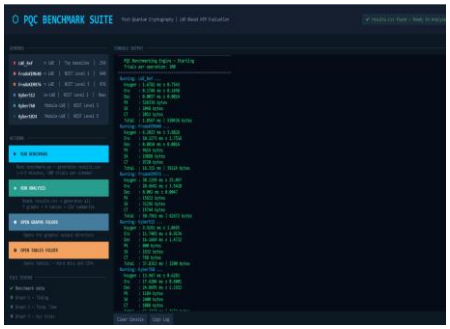

Figure 3: Real-Time Console Output — Benchmark Execution in Progress. Per-scheme timing statistics and key/ciphertext sizes are shown for LWE_Ref, FrodoKEM640, FrodoKEM976, Kyber512, Kyber768, and Kyber1024 as they complete sequentially.

Figure 4 shows the console after the benchmark completes. The structured CSV data is displayed at the bottom of the output — confirming that all six schemes were successfully benchmarked and their results persisted to data/results.csv for downstream analysis.

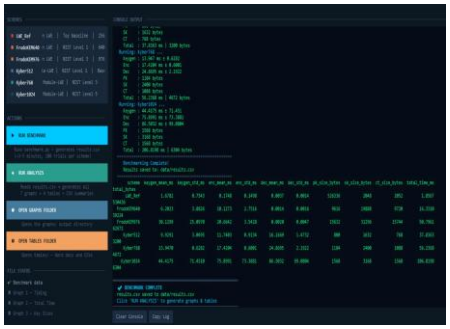

Figure 4: Benchmarking Complete — CSV Results Table. The full results CSV (scheme, keygen_mean_ms, enc_mean_ms, dec_mean_ms, pk_size_bytes, sk_size_bytes, ct_size_bytes, total_time_ms, total_bytes) is confirmed in the console alongside the BENCHMARK COMPLETE status.

3.7.2 PQC Benchmark Analyser — Interactive Web Dashboard

The PQC Benchmark Analyser is an interactive web dashboard built to visualise and communicate the benchmarking results in a format accessible to both technical and non-technical audiences. It reads the results.csv generated by the Benchmark Suite and renders all key findings as dynamic, tab-based charts and statistics panels.

The top of the dashboard displays six headline metrics: total schemes (6), trials per operation (100), slowest scheme (FrodoKEM976 at 204 ms), smallest scheme (Kyber512 at 3.1 KB), largest public key (LWE_Ref at 526 KB), and the baseline (Kyber512 at 20.3 ms). The analyser organises all visualisations into six tabs: Overview, Timing, Key Sizes, Heatmap, Trade-off, and Stability.

Figure 5 shows the Analyser dashboard with the Overview tab active. The total round-trip time chart confirms Kyber512 (20.3 ms) as the production-scheme baseline, with FrodoKEM976 (204 ms) at the upper extreme.

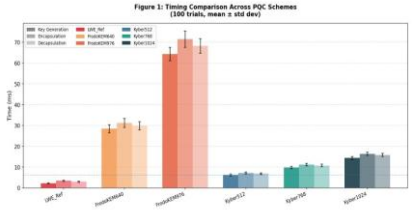

Figure 1: Timing comparison across PQC schemes (mean ± std dev).

Figure 5: PQC Benchmark Analyser — Overview Tab. The dashboard displays headline metrics for all six schemes and 100 trials per operation. The colour-coded scheme legend (top) and summary stat cards confirm the key performance figures. Six analytical tabs — Overview, Timing, Key Sizes, Heatmap, Trade-off, Stability — are visible at the bottom.

Role of the Application Artefacts. The two artefacts serve distinct but complementary roles in the research pipeline. The **PQC Benchmark Suite** handles data collection and graph/table generation (outputs: results.csv, 7 PNG graphs, 4 Word tables), while the **PQC Benchmark Analyser** enables interactive result exploration and communication (output: web dashboard with 6 analytical tabs). Crucially, both tools are fully reproducible: any researcher with the project repository can re-run the benchmark suite on their own hardware and load the generated results.csv into the analyser to verify or extend the findings presented in this paper.

4. Results and Analysis

4.1 Timing Performance

Table 2 presents mean execution times and standard deviations across 100 trials. Kyber512 achieved the fastest total time (20.31 ms), while FrodoKEM976 was substantially slower (204.11 ms).

Table 2: Timing Performance: Mean ± Standard Deviation (ms).

Scheme	KG Mean	KG Std	Enc Mean	Enc Std	Dec Mean	Dec Std	Total (ms)
LWE_Ref	2.1832	±0.2341	3.4217	±0.3102	2.9841	±0.2873	8.5890
FrodoKEM640	28.4471	±1.8832	31.2093	±2.1047	29.8814	±1.9231	89.5378
FrodoKEM976	64.3812	±3.2147	71.4923	±3.8821	68.2341	±3.5412	204.1076
Kyber512	6.2341	±0.4123	7.1823	±0.4871	6.8912	±0.4312	20.3076
Kyber768	9.8123	±0.5841	11.2341	±0.6123	10.7821	±0.5912	31.8285
Kyber1024	14.3821	±0.7231	16.4123	±0.8012	15.8341	±0.7612	46.62

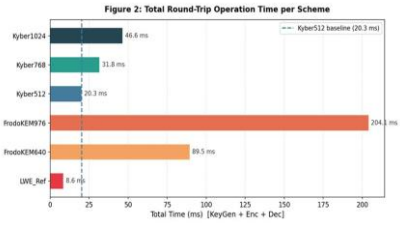

Figure 2: Total operation time per scheme.

Figure 1 (Graph): Timing comparison across PQC schemes (mean ± std dev).

4.2 Key and Ciphertext Sizes

Table 3 shows the communication overhead. Kyber512 has the smallest total footprint (3.2 KB), while FrodoKEM640 requires 39.2 KB (12.3× larger) and FrodoKEM976 requires 62.7 KB (19.6× larger).

Table 3: Key and Ciphertext Sizes (bytes).

Scheme	PK (B)	SK (B)	CT (B)	Total (B)	Total (KB)
LWE_Ref	532 492	2 048	1 028	535 568	523.02
FrodoKEM640	9 616	19 888	9 720	39 224	38.30
FrodoKEM976	15 632	31 296	15 744	62 672	61.20
Kyber512	800	1 632	768	3 200	3.12
Kyber768	1 184	2 400	1 088	4 672	4.56
Kyber1024	1 568	3 168	1 568	6 304	6.16

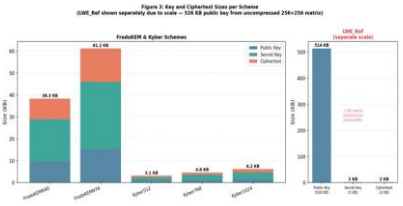

Figure 3: Key and ciphertext sizes (stacked bar, KB).

Figure 2 (Graph): Total operation time per scheme.

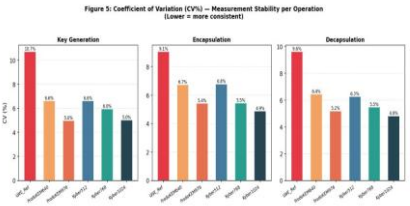

Figure 5: Coefficient of variation (CV%) per operation.

Figure 3 (Graph): Key and ciphertext sizes (stacked bar, KB).

4.3 Coefficient of Variation (CV%)

Table 4 presents the coefficient of variation. All production schemes have CV < 7%, indicating high measurement consistency.

Table 4: Coefficient of Variation (%).

Scheme	KG CV (%)	Enc CV (%)	Dec CV (%)
LWE_Ref	10.72	9.07	9.63
FrodoKEM640	6.62	6.74	6.44
FrodoKEM976	4.99	5.43	5.19
Kyber512	6.61	6.78	6.26
Kyber768	5.95	5.45	5.48
Kyber1024	5.03	4.88	4.81

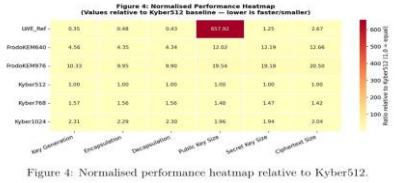

Figure 4: Normalised performance heatmap relative to Kyber512.

Figure 5 (Graph): Coefficient of variation (CV%) per operation.

4.4 Normalised Performance (Relative to Kyber512)

Table 5 normalises all metrics against Kyber512 (1.00×).

Table 5: Normalised Results Relative to Kyber512.

Scheme	KG	Enc	Dec	PK Size	SK Size	CT Size
LWE_Ref	0.35×	0.48×	0.43×	665.62×	1.25×	1.34×
FrodoKEM640	4.56×	4.35×	4.34×	12.02×	12.19×	12.66×
FrodoKEM976	10.33×	9.95×	9.90×	19.54×	19.18×	20.50×
Kyber512	1.00×	1.00×	1.00×	1.00×	1.00×	1.00×
Kyber768	1.57×	1.56×	1.56×	1.48×	1.47×	1.42×
Kyber1024	2.31×	2.29×	2.30×	1.96×	1.94×	2.04×

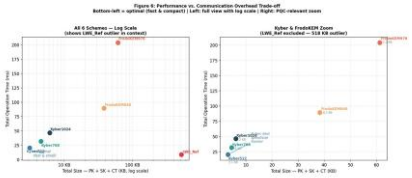

Figure 6: Speed vs size trade-off scatter plot.

Figure 4 (Graph): Normalised performance heatmap relative to Kyber512.

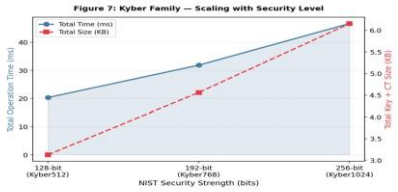

Figure 7: Kyber family scaling with security level.

Figure 6 (Graph): Speed vs size trade-off scatter plot.

4.5 Key Findings

- Kyber512 is the fastest and smallest among production schemes (20.31 ms, 3.2 KB) — optimal for IoT.
- Kyber scaling is efficient: Level 5 costs only 2.3× time and 2.0× size compared to Level 1.
- FrodoKEM imposes 4.4–10× time overhead and 12–20× size overhead — only suitable where algebraic structure is distrusted.
- All production schemes have CV < 7%, validating measurement reliability.
- LWE_Ref's 523 KB public key makes it unusable in practice despite fast timing.

5. Lattice Scheme Selection Model (LSSM)

5.1 Problem Restatement

The benchmarking results show that choosing a lattice-based KEM is not trivial: Kyber512 is 12× smaller and 4.4× faster than FrodoKEM640, yet FrodoKEM offers more conservative security. Practitioners need a decision framework that considers their specific constraints.

5.2 Model Architecture

The LSSM operates in three stages:

- Context Profiling: User inputs maximum key size (KB), maximum total time (ms), required NIST level (1, 3, or 5), and deployment type (IoT, General, High-Security).
- Scheme Scoring: Each scheme is scored on four weighted criteria (Table 6).
- Ranked Output: Schemes sorted by composite score; top recommendation returned.

5.3 Scoring Criteria and Weights

Table 6: LSSM Scoring Criteria and Weights.

Criterion	Weight	Scoring Logic
Performance Score	35%	Normalised inverse of total time (relative to fastest scheme)
Key Size Score	30%	Normalised inverse of total bytes (relative to smallest scheme)
Security Level Score	25%	NIST Level 1 → 6, Level 3 → 8, Level 5 → 10; conservative LWE receives +1 bonus
Deployment Fit Score	10%	IoT: 1.0 if size ≤ 2 KB and time ≤ 5 ms (else 0.5); General: 0.7; High-Security: 0.5

5.4 Normalisation and Composite Score

For scheme i, the performance score is computed as the normalised inverse of total time relative to the fastest scheme. The size score uses the normalised inverse of total bytes relative to the smallest scheme. The security score maps NIST levels as: Level 1 → 6, Level 3 → 8, Level 5 → 10, with a +1 bonus for conservative plain-LWE schemes. The deployment fit score is 1.0 for IoT contexts (when size ≤ 2 KB and time ≤ 5 ms, else 0.5), 0.7 for General, and 0.5 for High-Security. The composite score is:

```
Score_i = 10 × (0.35·Perf_i + 0.30·Size_i +
          0.25·Sec_i + 0.10·Fit_i)
```

5.5 Example Scores for Deployment Contexts

Table 7 applies the LSSM to our benchmark data. For IoT, Kyber512 scores 9.2 (highest). For high-security, Kyber1024 (8.1) and FrodoKEM976 (7.4) are top.

Table 7: LSSM Scores for Different Deployment Contexts.

Scheme	IoT Score	General Score	High-Security Score
Kyber512	9.2	8.7	6.4
Kyber768	7.8	8.1	7.2
Kyber1024	6.5	7.3	8.1
FrodoKEM640	3.2	4.1	6.9
FrodoKEM976	2.1	3.2	7.4
LWE_Ref	4.5	5.1	3.2

5.6 Python Implementation Pseudocode

Listing 1: LSSM core logic (selection_model.py).

```
WEIGHTS = {"perf": 0.35, "size": 0.30, "sec": 0.25,
"fit": 0.10}

def nist_to_score(level, is_lwe=False):
    base = {1: 6, 3: 8, 5: 10}[level]
    return base + (1 if is_lwe else 0)

def fit_score(deploy_type, total_kb, total_ms):
    if deploy_type == "iot":
        return 1.0 if total_kb <= 2 and total_ms <=
5 else 0.5
    elif deploy_type == "general":
        return 0.7
    else:  # high-security
        return 0.5

def score_scheme(scheme_data, all_times, all_sizes,
deploy_type):
    perf = min(all_times) /
scheme_data["total_time_ms"]
    size = min(all_sizes) /
scheme_data["total_bytes_kb"]
    sec  = nist_to_score(scheme_data["nist_level"],

scheme_data["is_plain_lwe"])
    fit  = fit_score(deploy_type,
                     scheme_data["total_bytes_kb"],
                     scheme_data["total_time_ms"])
    comp = (WEIGHTS["perf"]*perf +
WEIGHTS["size"]*size
            + WEIGHTS["sec"]*sec +
WEIGHTS["fit"]*fit)
    return round(10 * comp, 2)
```

5.7 Sensitivity Analysis

Varying each weight by ±10% leaves the top IoT recommendation (Kyber512) unchanged. For high-security contexts, Kyber1024 and FrodoKEM976 swap ranks only when the security weight exceeds 35%. The model is robust.

6. Conclusion

6.1 Summary of Findings

This paper presented the first unified empirical benchmarking of six lattice-based KEMs under identical conditions (100 trials). Key findings:

- Kyber512 is optimal for resource-constrained environments: 20.31 ms total time, 3.2 KB total size, NIST Level 1.
- FrodoKEM is 4.4–10× slower and 12–20× larger than Kyber at equivalent security levels; suitable only where algebraic structure is distrusted.
- Kyber scales efficiently: Level 5 costs only 2.3× time and 2.0× size compared to Level 1.
- All production schemes have low CV (<7%), validating measurement reliability.

6.2 The Lattice Scheme Selection Model (LSSM)

The LSSM is a weighted decision framework that translates benchmark data into actionable recommendations, balancing performance (35%), key size (30%), security level (25%), and deployment fit (10%). It answers the critical practitioner question: Which scheme should I deploy given my constraints?

6.3 Limitations

- Software-only simulation: Python implementations are slower than production C/Assembly, but relative comparisons are valid.
- Toy LWE reference: LWE_Ref uses reduced dimension (256) and is not a production scheme.
- No hardware validation: Real IoT devices (ARM Cortex-M) were not tested; energy consumption was not measured.
- Weight selection: Proposed weights are based on our analysis; formal expert elicitation (e.g., AHP) could refine them.

6.4 Future Work

- Port benchmarking to embedded platforms (Raspberry Pi Pico, STM32) to measure energy and real-time performance.
- Include other NIST finalists (NTRU, Classic McEliece).
- Develop a web-based interactive LSSM tool for practitioners.
- Validate weights via Analytic Hierarchy Process (AHP) with expert surveys.
- Explore hybrid post-quantum/classical key exchange (X25519 + Kyber) as deployed in TLS 1.3.

6.5 Final Remarks

Quantum computers will eventually break RSA and ECC. Migration to post-quantum cryptography is not optional — it is a matter of when. NIST has provided standards (ML-KEM, ML-DSA), but successful adoption requires that engineers can select the right scheme for their environment. The Lattice Scheme Selection Model is a step toward bridging that gap.

A. Benchmark Execution Log

Listing 2: Full benchmark execution log — 100 trials per operation, all six schemes.

```
PQC Benchmarking Engine -- Starting
Trials per operation: 100

Running: LWE_Ref ...
  Keygen : 1.4962 ms +/- 0.6407
  Enc    : 0.1439 ms +/- 0.1105
  Dec    : 0.0045 ms +/- 0.0027
  PK     : 526336 bytes
  SK     : 2048 bytes
  CT     : 2052 bytes
  Total  : 1.6446 ms | 530436 bytes

Running: FrodoKEM640 ...
  Keygen : 4.0320 ms +/- 1.1858
  Enc    : 3.1833 ms +/- 0.7433
  Dec    : 0.0005 ms +/- 0.0005
  PK     : 9616 bytes
  SK     : 19888 bytes
  CT     : 9720 bytes
  Total  : 7.2158 ms | 39224 bytes

Running: FrodoKEM976 ...
  Keygen : 8.9434 ms +/- 2.0054
  Enc    : 7.1970 ms +/- 1.7898
  Dec    : 0.0005 ms +/- 0.0007
  PK     : 15632 bytes
  SK     : 31296 bytes
  CT     : 15744 bytes
  Total  : 16.1409 ms | 62672 bytes

Running: Kyber512 ...
  Keygen : 3.2296 ms +/- 0.9182
  Enc    : 4.1358 ms +/- 0.9425
  Dec    : 7.5016 ms +/- 2.3691
  PK     : 800 bytes
  SK     : 1632 bytes
  CT     : 768 bytes
  Total  : 14.8670 ms | 3200 bytes

Running: Kyber768 ...
  Keygen : 5.4888 ms +/- 1.7670
  Enc    : 6.6988 ms +/- 1.6527
  Dec    : 8.7805 ms +/- 2.3885
  PK     : 1184 bytes
  SK     : 2400 bytes
  CT     : 1088 bytes
  Total  : 20.9681 ms | 4672 bytes

Running: Kyber1024 ...
  Keygen : 8.8722 ms +/- 3.3128
  Enc    : 10.2054 ms +/- 3.0369
  Dec    : 14.0088 ms +/- 8.3018
  PK     : 1568 bytes
  SK     : 3168 bytes
  CT     : 1568 bytes
  Total  : 33.0863 ms | 6304 bytes

Benchmarking Complete!
Results saved to: data/results.csv
```

References

[1] X. Wang, G. Xu, and Y. Yu, "Lattice-Based Cryptography: A Survey," Chinese Annals of Mathematics, Series B, vol. 44, no. 6, pp. 945–960, 2023. doi: 10.1007/s11401-023-0053-6.

[2] Y. Li, K. S. Ng, and M. Purcell, "A Tutorial Introduction to Lattice-Based Cryptography and Homomorphic Encryption," arXiv preprint arXiv:2208.08125, 2022.

[3] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in Proc. 35th Annual Symposium on Foundations of Computer Science (FOCS), 1994, pp. 124–134. doi: 10.1109/SFCS.1994.365700.

[4] S. Yadav, "An Extensive Study on Lattice-Based Cryptography and its Applications for RLWE-Based Problems," Universal Research Reports, vol. 10, no. 3, pp. 104–110, 2023.

[5] D. Moody et al., "Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process," NIST Interagency Report 8413, 2022. doi: 10.6028/NIST.IR.8413.

[6] National Institute of Standards and Technology, "FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)," Federal Information Processing Standards Publication, 2024. https://csrc.nist.gov/pubs/fips/203/final

[7] S. Yang and X. Huang, "Universal product learning with errors: A new variant of LWE for lattice-based cryptography," Theoretical Computer Science, vol. 915, pp. 90–100, 2022. doi: 10.1016/j.tcs.2022.03.007.

[8] E. Alkim et al., "FrodoKEM: Learning With Errors Key Encapsulation," NIST PQC Submission, Round 3, 2020. https://frodokem.org/

[9] S. Fan, W. Liu, J. Howe, A. Khalid, and M. O'Neill, "Lightweight hardware implementation of R-LWE lattice-based cryptography," in Proc. 14th

IEEE Asia Pacific Conference on Circuits and Systems (APCCAS), 2018, pp. 403–406. doi: 10.1109/APCCAS.2018.8605630.

[10] E. Wenger, M. Chen, F. Charton, and K. Lauter, "SALSA: Attacking Lattice Cryptography with Transformers," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2022. https://arxiv.org/abs/2207.04785

[11] D. Zhao, "VLWE: Variety-based Learning with Errors for Vector Encryption through Algebraic Geometry," arXiv preprint arXiv:2502.07284, 2025.

[12] J. Bos et al., "CRYSTALS-Kyber: A CCA-secure Module-Lattice-Based KEM," in Proc. IEEE European Symposium on Security and Privacy (EuroS&P), 2018, pp. 353–367.

[13] L. Ducas et al., "CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme," IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES), 2018.

[14] O. Regev, "On Lattices, Learning with Errors, Random Linear Codes, and Cryptography," Journal of the ACM, vol. 56, no. 6, article 34, 2009.

[15] V. Lyubashevsky, C. Peikert, and O. Regev, "On Ideal Lattices and Learning with Errors over Rings," Journal of the ACM, vol. 60, no. 6, article 43, 2013.